

Fundamentals of Algorithms and Data Structures

Lecture 4 – Nonlinear Data Structures: Trees

Prof. André Gustavo Hochuli

gustavo.hochuli@pucpr.br

aghochuli@ppgia.pucpr.br

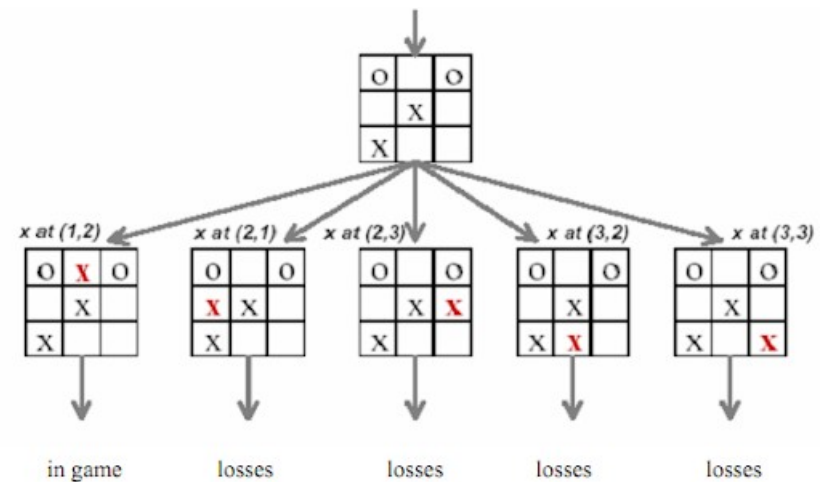
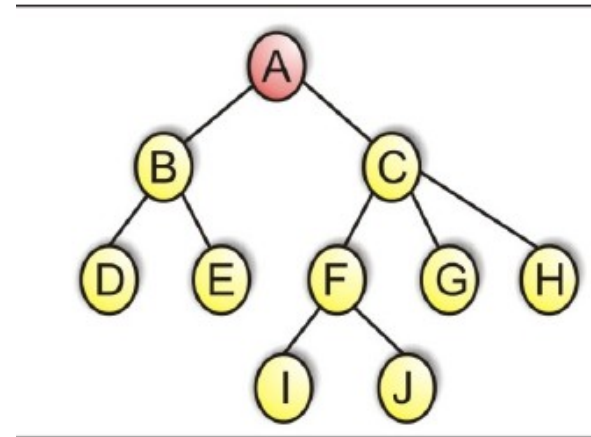
Outline

- Nonlinear Data Structures
 - Tree Concepts
 - Binary Trees
 - Balanced Binary Trees (AVL – Adelson-Velsky and Landis)

Trees

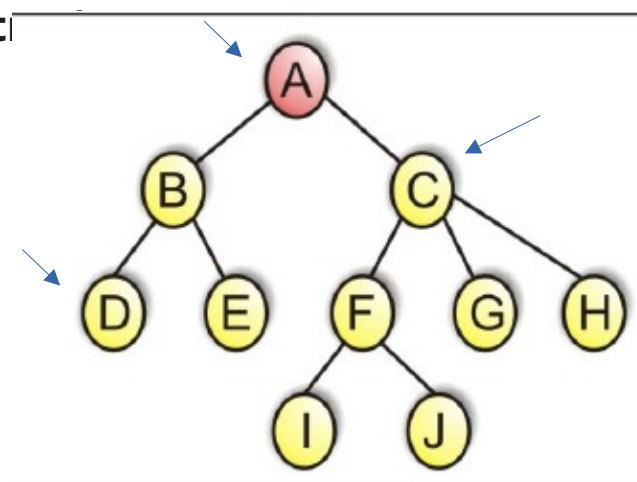
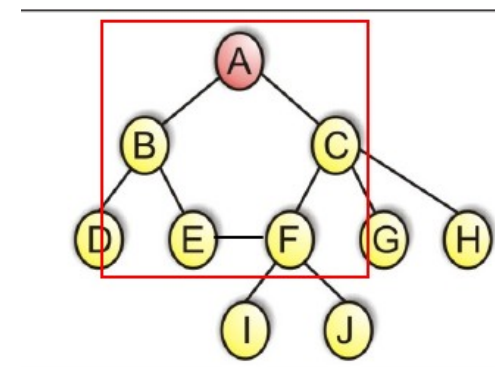
Trees

- Nonlinear Data Structures
- Hierarchical Representation
- Applications
 - Syntax Checkers(Grammars, Compilers)
 - Databases
 - Routers
 - Process Schedulers
 - Artificial Intelligence (AI)



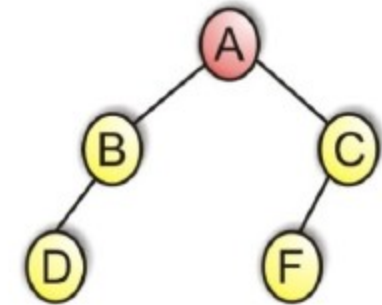
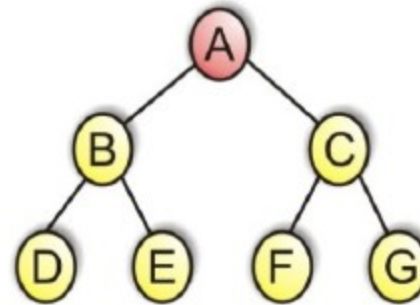
Tree Data Structures – Core Concepts

- **Trees Do Not Contain Cycles**
- **Degree**
 - Number of subtrees (i.e., number of children of a node)
 - Nodes: A = 2, C = 3, D = 0
- **Tree Degree: 3** (maximum node degree in the tree)
- **Level**
 - Distance from a node to the root
 - Nodes: D = 2, I = 3
- **Tree Level (Height): 3**



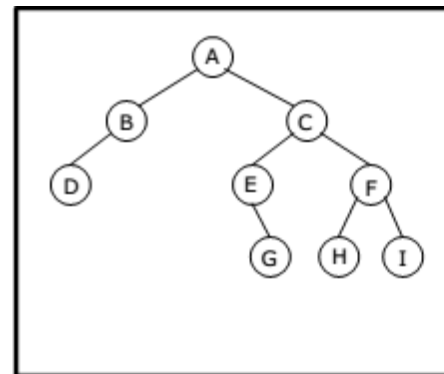
Binary Tree Structures

- Binary Trees Have a Maximum Degree of 2



- Tree Traversal (Visitation Strategy)

- Preorder: root → left → right
- Postorder: left → right → root
- Inorder: left → root → right
- Level-order*: level by level ($N = 0 \rightarrow N = 1 \rightarrow \dots$)



- Preorder
A, B, D, C, E, G, F, H, I
- Postorder
D, B, G, E, H, I, F, C, A
- Inorder
D, B, A, E, G, C, H, F, I
- Level order
A, B, C, D, E, F, G, H, I

- (*) Non-recursive method
- (*) Requires a queue (tree traversal using a queue)

Binary Tree - Implementation

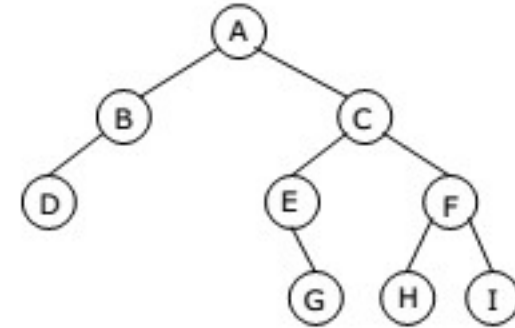
```
class Node:
    def __init__(self, data):
        self.data = data # Assign data
        self.left = None # Initialize as None
        self.right = None # Initialize as None

class Binary_Tree:
    # Init Class
    def __init__(self, data):
        self.root = Node(data)

    def push(self, data):
        if self.root is None:
            print("Root")
            self.root = Binary_Tree(data)

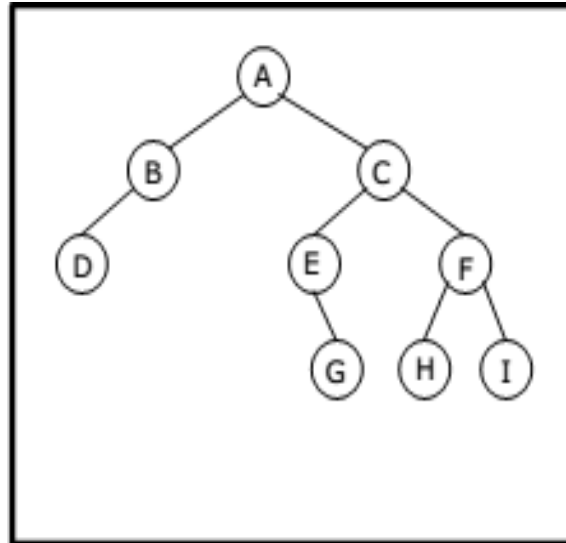
        if data > self.root.data:
            if self.root.right is None:
                print("Add Right")
                self.root.right = Binary_Tree(data)
            else:
                self.root.right.push(data)
        else:
            if self.root.left is None:
                print("Add Left")
                self.root.left = Binary_Tree(data)
            else:
                self.root.left.push(data)

    return
```



Trees - Transversal

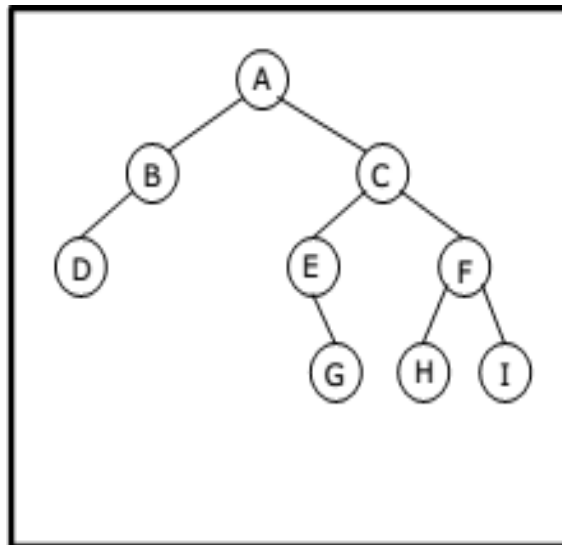
```
def walk_preorder(self):  
    print(self.root.data)  
    if self.root.left is not None:  
        self.root.left.walk_preorder()  
    if self.root.right is not None:  
        self.root.right.walk_preorder()
```



- Preorder
A, B, D, C, E, G, F, H, I
- Postorder
D, B, G, E, H, I, F, C, A
- Inorder
D, B, A, E, G, C, H, F, I
- Level order
A, B, C, D, E, F, G, H, I

Trees - Exercise

- Implement the following tree traversals:
 - Post-Order Transversal
 - In-Order Transversal



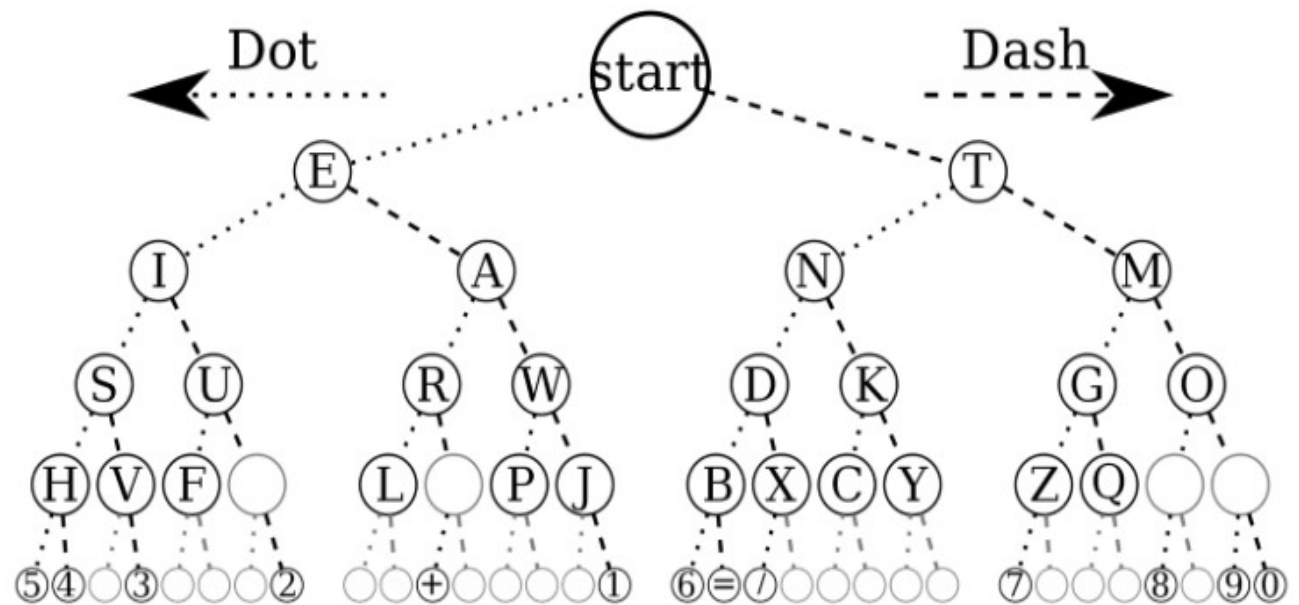
- Preorder
A, B, D, C, E, G, F, H, I
- Postorder
D, B, G, E, H, I, F, C, A
- Inorder
D, B, A, E, G, C, H, F, I
- Level order
A, B, C, D, E, F, G, H, I

Trees – Exercise

Implementing a Morse Code Translator

A	..	J		S	...	2	
B		K		T	-	3	
C		L		U	...	4	
D	...	M	--	V		5	
E	.	N	--	W	---	6	
F		O	---	X		7	
G	---	P		Y		8	
H		Q		Z		9	
I	..	R	...	1		0	

--- / - - . - . . . - = Ola Mundo



AVL Trees (Adelson-Velsky and Landis Trees)

AVL Balanced Binary Trees

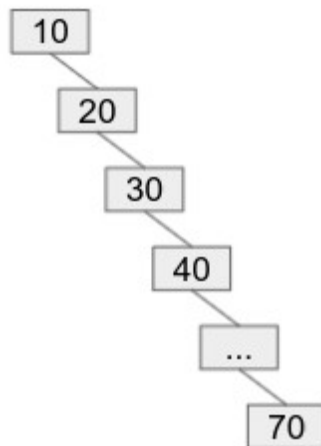
- **Adelson Velsky e Landis (AVL)**



Adelson-Velsky, Georgy; Landis, Evgenii (1962). "An algorithm for the organization of information". Proceedings of the USSR Academy of Sciences (in Russian). 146: 263–266

- **Insert the following keys sequentially into a Binary Search Tree (BST):**

- **10,20,30,40,50,60,70**



Causes Tree Imbalance

AVL Balanced Binary Trees

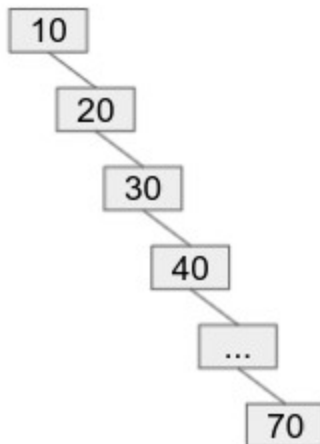
- **Adelson Velsky e Landis (AVL)**



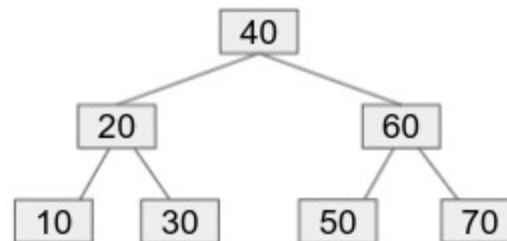
Adelson-Velsky, Georgy; Landis, Evgenii (1962). "An algorithm for the organization of information". Proceedings of the USSR Academy of Sciences (in Russian). 146: 263–266

- **Insert the following keys sequentially into a Binary Search Tree (BST):**

- **10,20,30,40,50,60,70**



Unbalanced Tree



AVL Balanced Binary Tree

AVL Balanced Binary Trees

- **Balanced Tree:** for every node, the height difference between the left and right subtrees is at most ± 1 .

- That is, the Balance Factor is defined as:

- $BF(n) = h_l - h_r \rightarrow |BF(n)| \leq 1 \Rightarrow \text{node is balanced}$

- $BF(n) = 0 \rightarrow h_l = h_r$

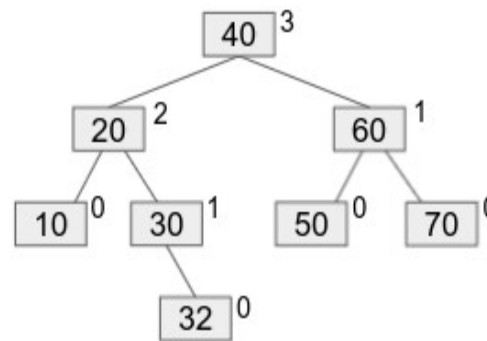
- $BF(n) > 0 \rightarrow h_l > h_r$

- $BF(n) < 0 \rightarrow h_l < h_r$

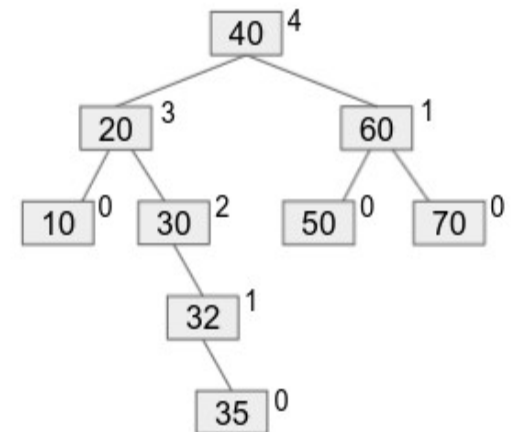
- **Notation:**

- (h_l) : height of the left subtree

- (h_r) : height of the right subtree

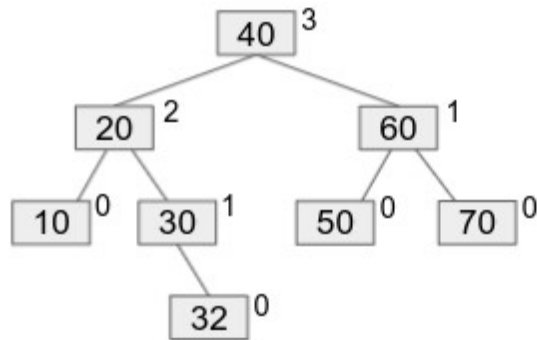


Balanced Tree



Unbalanced Tree

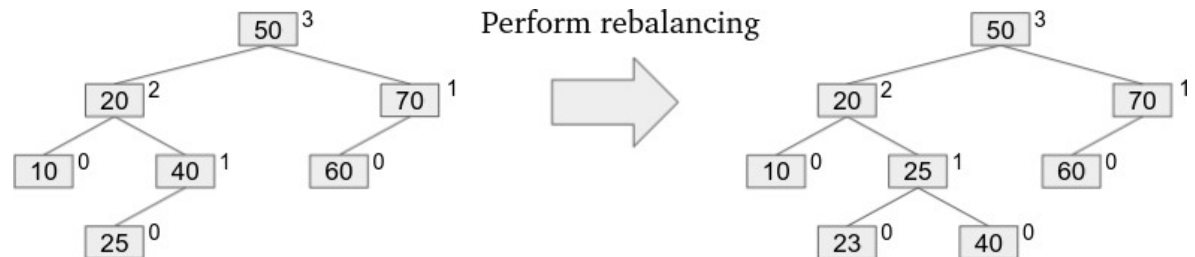
AVL Balanced Binary Trees



```
class Node:
    def __init__(self, data):
        self.data = data # Assign data
        self.left = None # Initialize as None
        self.right = None # Initialize as None
        self.height = 1
```

AVL Balanced Binary Trees

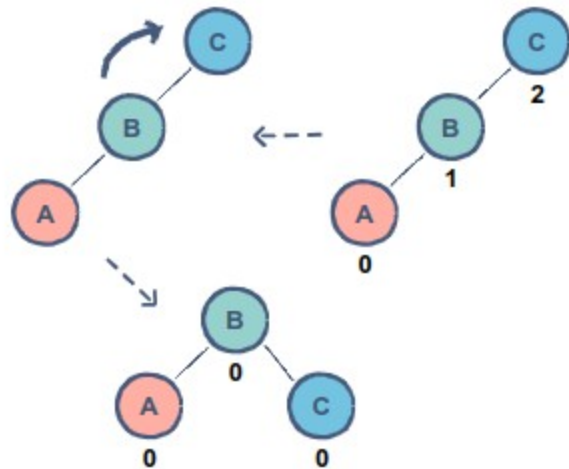
- Insertion Step:
 - Insert key 23
 - Perform rebalancing if $|BF| > 1$
- How to Determine Rotations (AVL)?
 - RR → Right Rotation
 - LL → Left Rotation
 - LR → Left + Right
 - RL → Right + Left



AVL Balanced Binary Trees

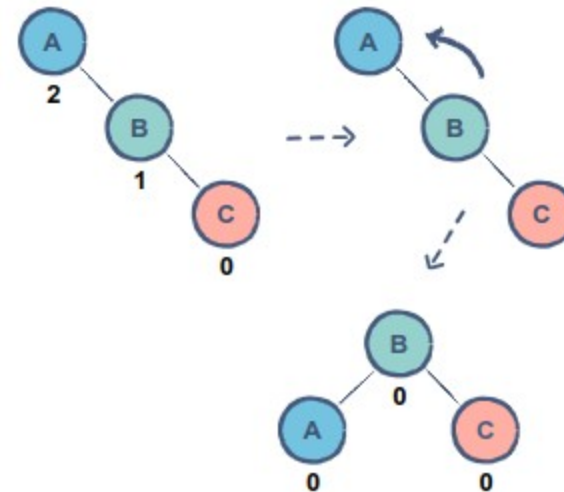
- Right Rotation (RR)

- $BF > 1$ and insertion occurs in the left subtree of left child



- Left Rotation (LL)

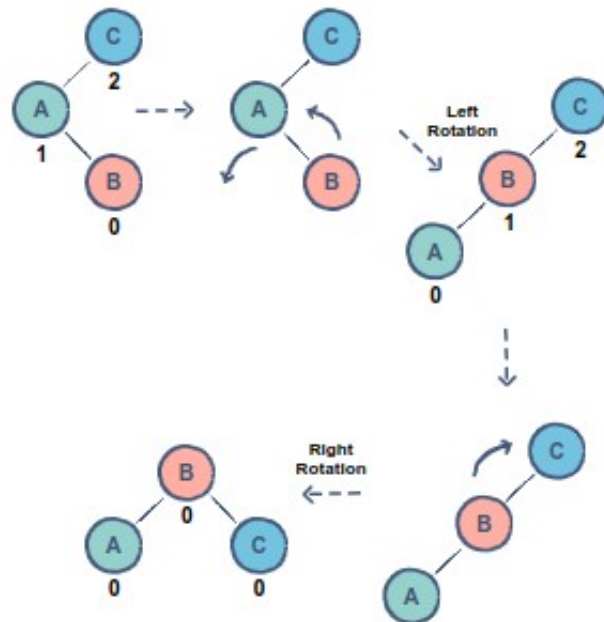
- $BF < -1$ and the inserted value is in the right subtree of right child



AVL Balanced Binary Trees

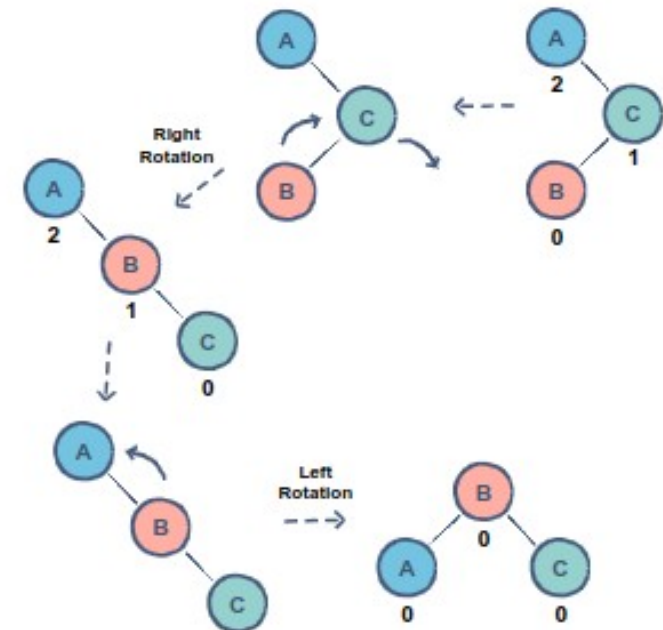
- Double Rotation LR :

- Left (on child) → Right (on node)
- $BF > 1$ e insertion occurs in the right subtree of the left child



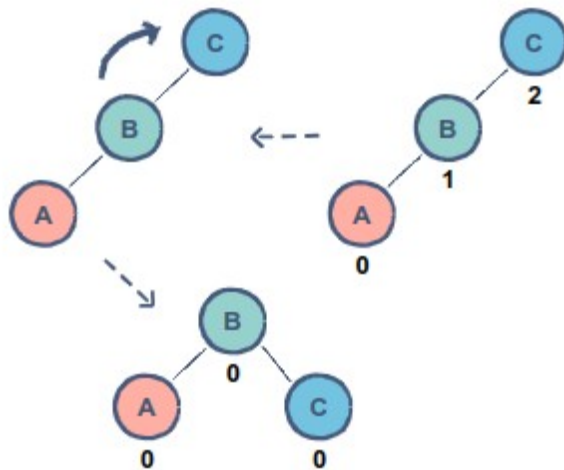
- Double Rotation RL :

- Right (on child) → Left (on node)
- $BF > -1$ e insertion occurs in the right subtree of the left child



AVL Balanced Binary Trees

- Right Rotation (RR)
- $BF > 1$ and insertion occurs in the left subtree of left child



```
Node *rightRotate(Node *y)
{
    Node *x = y->left;
    Node *T2 = x->right;

    // Perform rotation
    x->right = y;
    y->left = T2;

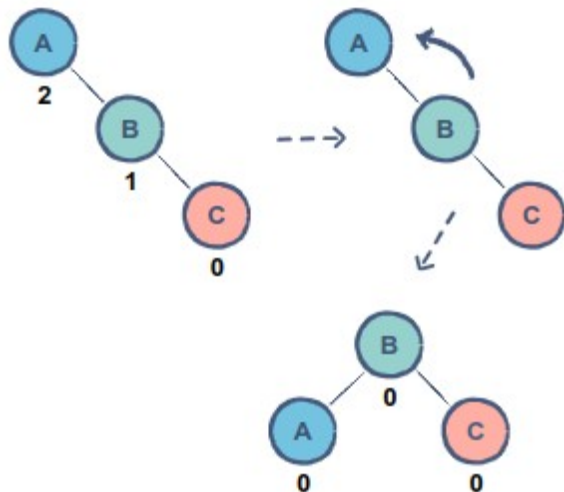
    // Update heights
    y->height = max(height(y->left),
                    height(y->right)) + 1;
    x->height = max(height(x->left),
                    height(x->right)) + 1;

    // Return new root
    return x;
}
```

```
if (balance > 1 && key < node->left->key)
    node = rightRotate(node);
```

AVL Balanced Binary Trees

- Left Rotation (LL)
- $BF < -1$ and the inserted value is in the right subtree of right child



```
Node *leftRotate(Node *x)
{
    Node *y = x->right;
    Node *T2 = y->left;

    // Perform rotation
    y->left = x;
    x->right = T2;

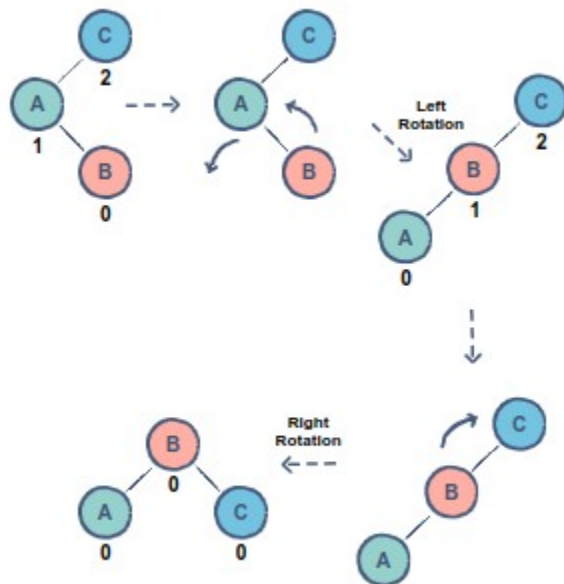
    // Update heights
    x->height = max(height(x->left),
                    height(x->right)) + 1;
    y->height = max(height(y->left),
                    height(y->right)) + 1;

    // Return new root
    return y;
}
```

```
if (balance < -1 && key > node->right->key)
    node = leftRotate(node);
```

AVL Balanced Binary Trees

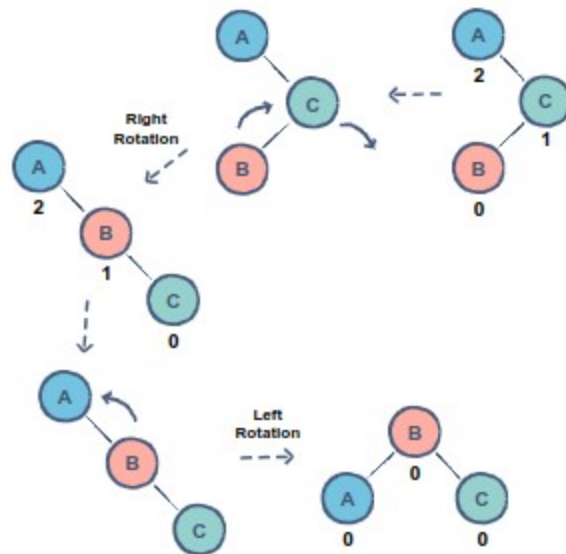
- LR (Double Rotation):
 - Left (on child) → Right (on node)
- $BF > 1$ and insertion occurs in the right subtree of the left child



```
if (balance > 1 && key > node->left->key)
{
    node->left = leftRotate(node->left);
    node = rightRotate(node);
}
```

AVL Balanced Binary Trees

- RL (Double Rotation):
 - Right (on child) → Left (on node)
- $BF < -1$ and insertion occurs in the left subtree of the right child



```
if (balance < -1 && key < node->right->key)
{
    node->right = rightRotate(node->right);
    return leftRotate(node);
}
```

AVL Balanced Binary Trees

- Exercise (Manual Practice)
- Insert the following sequences into both an AVL tree and a non-balanced binary search tree:
 - 10, 3, 2, 5, 7, 6
 - A, B, C, ..., J
- Then, for both trees, compute:
 - the inorder traversal
 - the number of comparisons/search steps needed to find 7 and H
 - the asymptotic time complexity (Big-O)
- Explain why the complexities differ between AVL and unbalanced trees.
- AVL Simulators:
 - <https://cmps-people.ok.ubc.ca/ylucet/DS/AVLtree.html>
 - <https://visualgo.net/en>

Assignment

Trabalho 01: Análise Comparativa de Estruturas de Dados

> Tarefas > Trabalho 01: Análise Comparativa de Estruturas de Dados

Mestrado

Página inicial

Avisos

→ Tarefas

Testes

Fóruns

Chat

Arquivos

Trabalho 01: Análise Comparativa de Estruturas de Dados

✓ Publicado Atribuir a Editar ⋮

O trabalho é formalizado no documento PDF abaixo anexado:

https://drive.google.com/file/d/1iNAsY1hraWFMaJjWwBbhne1AnEuk6IHr/view?usp=drive_link

!!! Groups must be created via Canvas !!!

!!! Students must be assigned to a group by the deadline to receive a grade !!!